

# Data Science

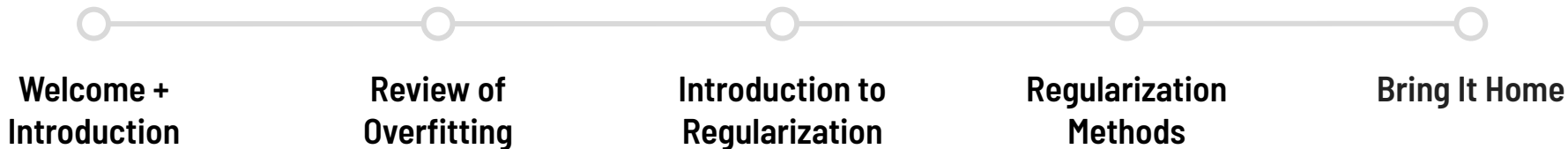
## Regularization

Machine Learning Fundamentals



GENERAL ASSEMBLY

# Lesson roadmap



# Learning objectives



1

Define and describe regularization for regression models.

2

Understand the causes of overfitting in linear models.

3

Describe how regularization effects regression coefficients.

4

Compare Ridge, Lasso, and ElasticNet regularization techniques.

5

Implement regularized models using **scikit-learn**.



# Review of Overfitting



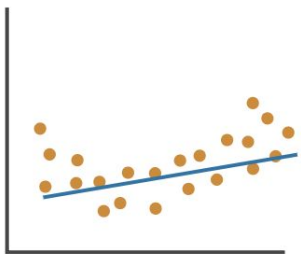
# Trivia

What is overfitting?

# Model Fit

## Underfit

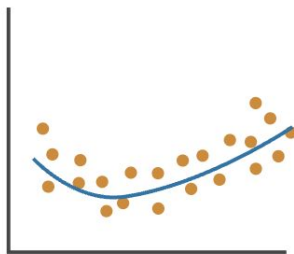
Doesn't capture the shape of the data (curve/ parabolic shape).



Training Data: High Error  
New Data: High Error

## Balanced Model

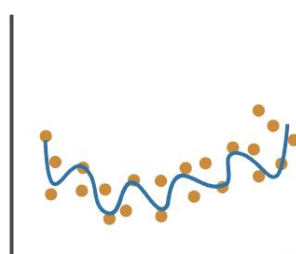
This model is a good balance that captures the training data well and will work well for new data.



Training Data: Medium Error  
New Data: Medium Error

## Overfit

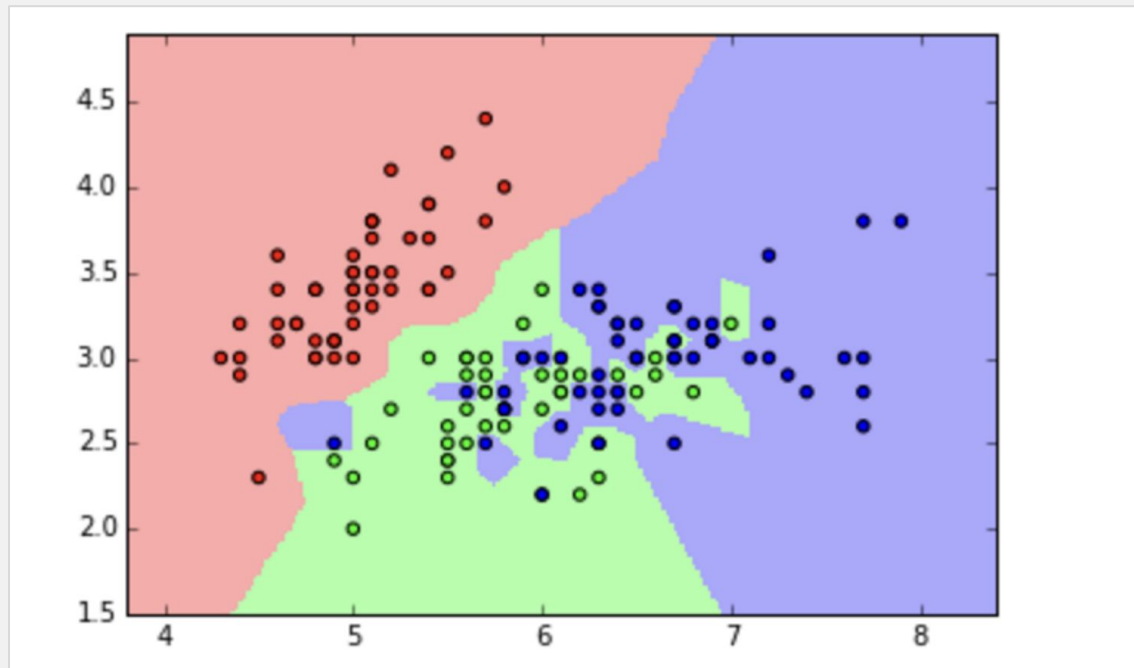
Memorizes each data point and won't predict new values accurately.



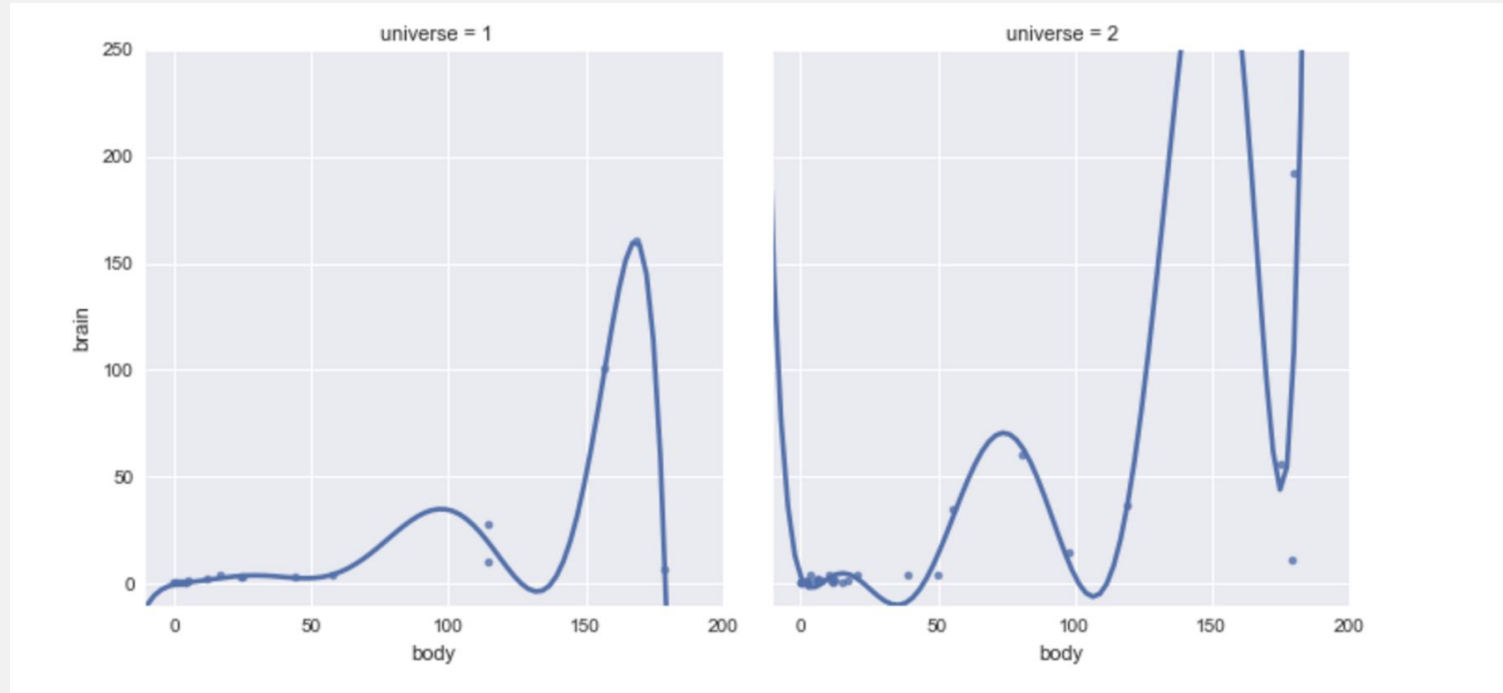
Training Data: Low Error  
New Data: High Error



# Overfitting Examples: kNN

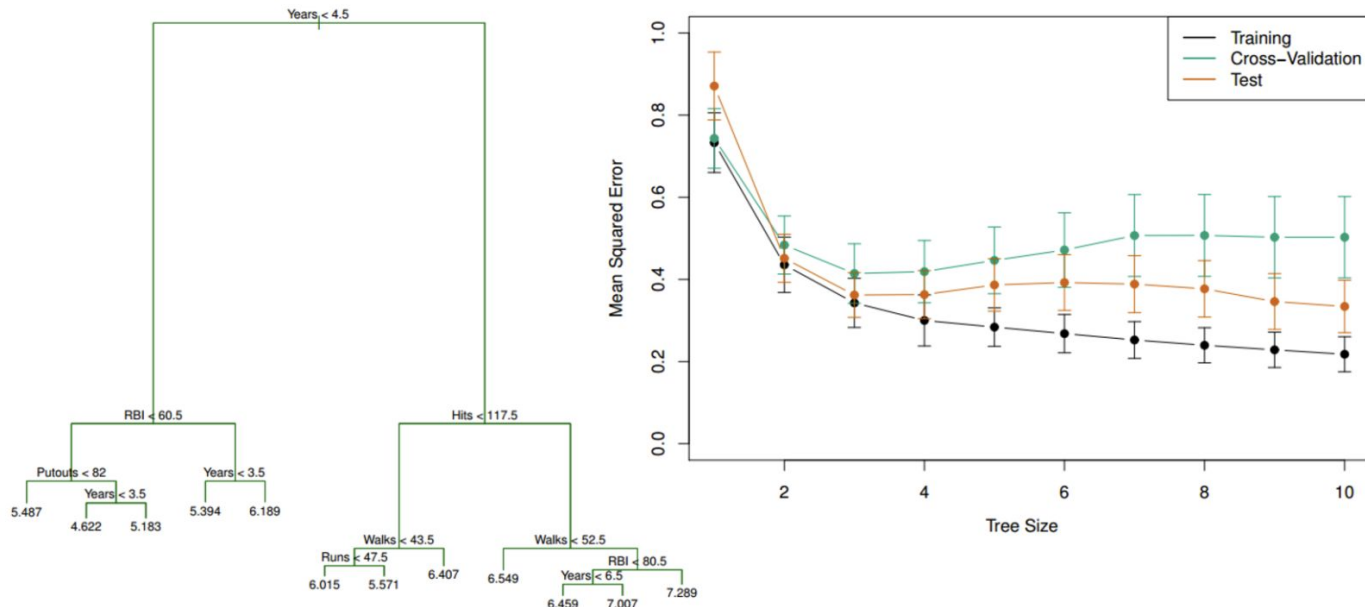


# Overfitting Examples: Polynomial Regression





## Overfitting Examples: Decision Trees



## Overfitting

→ Think about a linear model. How is it impacted by overfitting?

- How does overfitting occur?
- What is the impact?

# General Characteristics of Linear Models

1

Low model  
complexity.

2

High bias, low  
variance.

3

Does not tend to  
overfit.

Nevertheless, overfitting can still occur with linear models if you allow them to have high variance.

# Overfitting in Linear Models

1

Irrelevant  
features.

2

Correlated  
features.

3

Large  
Coefficients.



## Overfitting: Irrelevant Features

Linear models can overfit if you include **irrelevant features**, meaning features that are unrelated to the response. Why?

## Overfitting: Irrelevant Features

**Linear models can overfit if you include irrelevant features, meaning features that are unrelated to the response. Why?**

- Because the model will learn a coefficient for every feature you include in it, regardless of whether that feature has the signal or the noise.
- This is especially a problem when  $p$  (number of features) is close to  $n$  (number of observations), because that model will naturally have high variance.

## Overfitting: Correlated Features

Linear models can overfit if the included features are **highly correlated** with one another. Why?

## Overfitting: Correlated Features

Linear models can overfit if the included features are **highly correlated** with one another. Why?

→ From the **scikit-learn documentation**:

"...coefficient estimates for Ordinary Least Squares rely on the independence of the model terms. When terms are correlated and the columns of the design matrix  $X$  have an approximate linear dependence, the design matrix becomes close to singular and as a result, the least-squares estimate becomes highly sensitive to random errors in the observed response, producing a large variance." [Link](#)

## Overfitting: Large Coefficients

Linear models can overfit if the coefficients are too large, after feature standardization. Why?

## Overfitting: Large Coefficients

**Linear models can overfit if the coefficients are too large, after feature standardization. Why?**

- Because the larger the absolute value of the coefficient, the more power it has to change the predicted response, resulting in a higher variance.



---

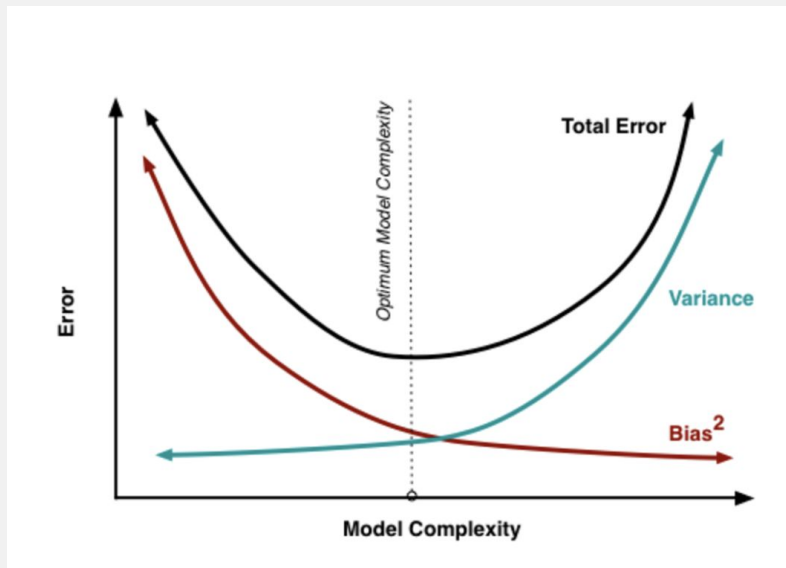
**Regularization can help  
address these issues!**

---

# Introduction to Regularization

# What Is Regularization?

Our goal is to locate the optimum model complexity, and thus **regularization is useful when we believe our model is too complex.**



# What Is Regularization?

1

A method for "constraining" or "regularizing" the size of coefficients.

2

Shrinks coefficients toward zero.

3

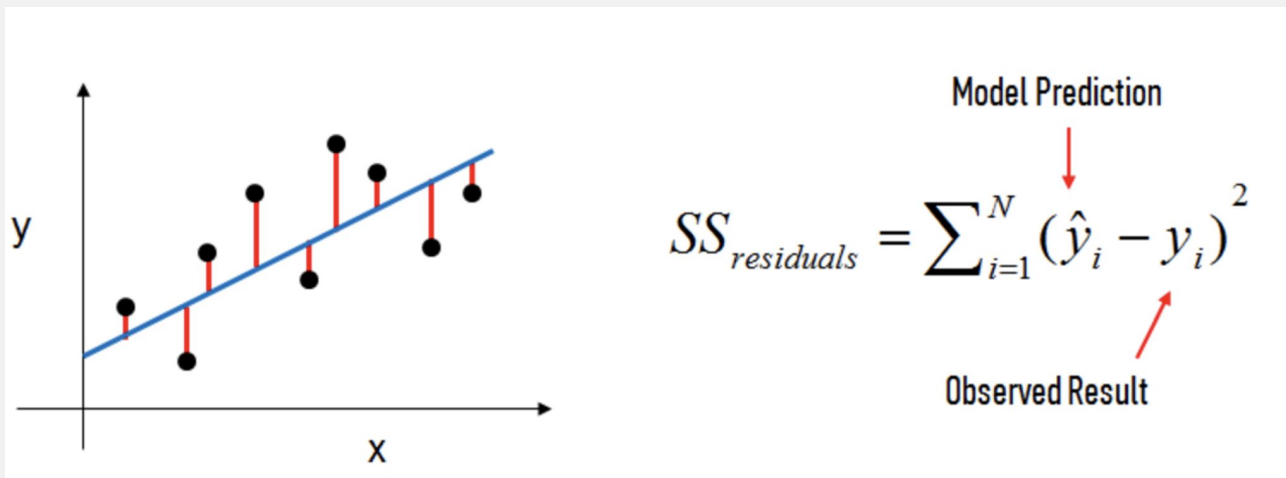
Reduces model variance and minimizes overfitting.

4

Tends to reduce variance more than it increases bias.

# Regularization Method

For standard linear regression, we minimize the residual sum of squares (RSS):



# Regularization Method

Regularization adds a **penalty term** to discourage large coefficients.

$$\begin{aligned} RSS(\beta_0, \beta_1) &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^n (\hat{y}_i - \beta_0 - \beta_1 x_i)^2 \end{aligned}$$

The goal of training is to minimize  $RSS(\beta_0, \beta_1)$ , i.e.

$$\beta_0, \beta_1 = \arg \min RSS(\beta_0, \beta_1)$$

# Regularization Methods

There are a variety of methods available to accomplish this.  
In this lesson, you will cover:

- Ridge
- Lasso
- Elastic Net



# Regularization Methods



# Ridge Regression (L2)

Ridge regression **penalizes** the model for having large coefficients. Ridge **shrinks** the regression coefficients.

$$J(\beta_0, \beta_1) = RSS(\beta_0, \beta_1) + \alpha\beta_1^2$$

As  **$\alpha$**  increases,  **$\beta_1$**  will decay. Where  **$\alpha$**  is our **tuning parameter**.

## Explore Ridge Regression

- Find the **scikit-learn** documentation on Ridge Regression.
- Locate the model description (inputs, outputs, parameters, methods), and the discussion of the theory with examples.

- ☐ How do you set the regularization strength?
- ☐ How can you get the values of the regularized regression coefficients?

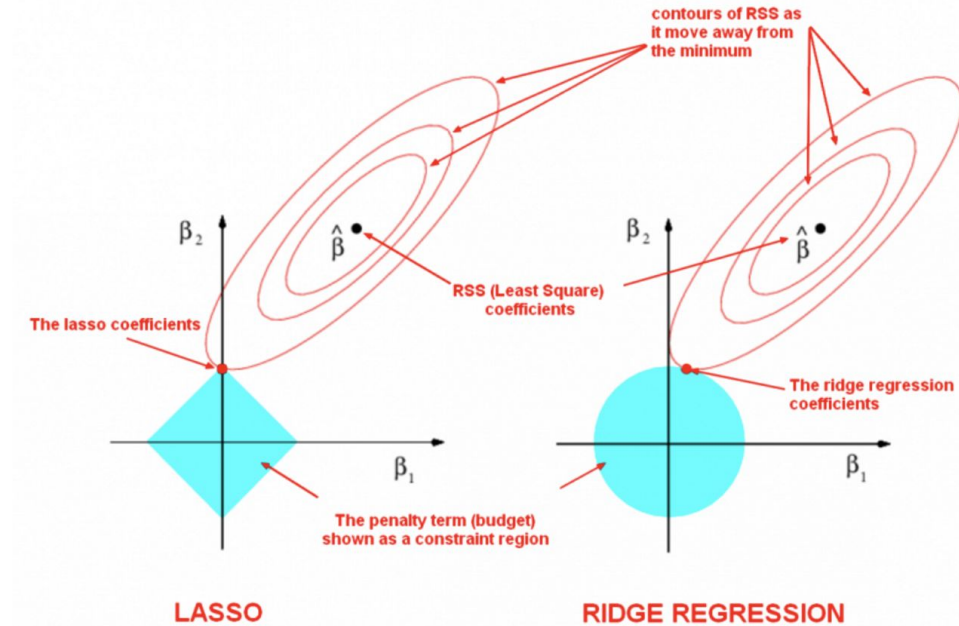
# Lasso Regression (L1)

Lasso regression **penalizes** the model for having large coefficients, all the way to zero if needed. Lasso **shrinks** the regression coefficients, and may zero out unimportant features.

$$J(\beta_0, \beta_1) = RSS(\beta_0, \beta_1) + \alpha |\beta_1|$$

As  **$\alpha$**  increases,  **$\beta_1$**  will decay. Where  **$\alpha$**  is our **tuning parameter**.

# Ridge (L2) and Lasso (L1) Comparison



# When to use Ridge or Lasso?

- **Lasso regression** is preferred if we believe many features are irrelevant or if we prefer a sparse model.
- **Ridge regression** is preferred if we have many relevant features which are correlated with each other.
- If model performance is your primary concern, it is best to try both Ridge and Lasso.

# ElasticNet

**ElasticNet regression** is a combination of lasso regression and ridge regression.

- Combines benefits of Ridge and Lasso.
- Can perform feature selection (like Lasso).
- Handles correlated features well (like Ridge).
- "Allows for learning a sparse model where few weights are non-zero like Lasso, while still maintaining the regularization properties of Ridge".

$$J(\beta_0, \beta_1, \dots, \beta_p) = RSS + \alpha \rho \sum |\beta_j| + \frac{\alpha(1-\rho)}{2} \sum \beta_j^2$$

**$\alpha$**  controls overall regularization strength  
 **$\rho$**  balances between L1 (Lasso) and L2 (Ridge) penalties

# Practical Considerations of Regularization

- Features should be standardized before applying regularization.
  - ◆ Otherwise, features would be penalized based on their scale, not important.
  - ◆ Standardization also prevents penalizing the intercept.
- Increase  $\alpha$  (more regularization):
  - ◆ Increase bias
  - ◆ Decrease variance
- Decrease  $\alpha$  (less regularization):
  - ◆ Decrease bias
  - ◆ Increase variance

## Ridge and Lasso

Let's see it for ourselves!

- Open **02-regularization.ipynb**





# Bring It Home





# Key takeaways

- Regularization is used when evidence suggests our model is suffering from high error due to variance.
- Evidence to suggest our model suffers from high error due to variance includes substantially better performance on our training set than our testing set.
- Lasso tends to be "more brutal" than Ridge regularization in that it will zero out coefficients.
- If you want to combine Lasso and Ridge regularization, ElasticNet that does exactly this.

